TP3 · Aide-mémoire Python

Normalisation, mots vides, racines et lemmes · TALN · EMSI · Chapitre 2 (partie 3)

Où trouver quoi ?

Partie du TP	Fiches utiles
0 · Préparation	1 (imports, assert, f-strings), 7 (spaCy, add_special_case), 10 (re.sub, html)
1 · Mots vides	2 (ensembles), 3 (compréhensions, sum), 4 (Counter), 7 et 8 (listes de mots vides)
2 · Racines et lemmes	8 (Snowball, Porter), 7 (lemma_, pos_), 5 (dictionnaires, setdefault), 9 (pandas)
3 · Arabe	10 (codes Unicode, re.sub, replace, translate)
4 · Vocabulaire	3 (listes de listes, ensembles), 6 (fonctions), 9 (pandas), 10 (unicodedata), 11 (barh)
Pour aller plus loin	8 (edit_distance, min sur des tuples)

Les exemples utilisent d'autres données que le TP : adaptez-les, ne les recopiez pas tels quels.

1. Préparer l'environnement

Ce qu'il faut exécuter une fois au début du notebook.

Installer et importer

Sur Colab, une ligne qui commence par `!` est une commande système. `import` charge un module ; `from module import nom` charge un seul élément.

```
# !pip install -q spacy
# !python -m spacy download fr_core_news_sm      # modèle français
# !python -m spacy download en_core_web_sm       # modèle anglais
import re, html, unicodedata
from collections import Counter
import pandas as pd
import matplotlib.pyplot as plt
```

Charger NLTK et spaCy

`nltk.download()` télécharge une ressource (ici la liste de mots vides). `spacy.load()` charge un modèle de langue : on obtient l'objet `nlp` qui analyse les textes.

```
import nltk
nltk.download("stopwords", quiet=True)
from nltk.corpus import stopwords
from nltk.stem.snowball import SnowballStemmer
from nltk.stem import PorterStemmer

import spacy
nlp_fr = spacy.load("fr_core_news_sm")
nlp_en = spacy.load("en_core_web_sm")
```

`assert` : vérifier une condition

Si la condition est vraie, rien ne se passe. Si elle est fausse, Python s'arrête avec `AssertionError` et affiche le message. C'est ce qu'utilisent les cellules ✅ **Test**.

```
assert 2 + 2 == 4, "calcul faux"      # vrai : rien ne s'affiche
NOM = ""
try:
    assert NOM, "Complétez NOM"        # "" est considéré comme faux
except AssertionError as e:
    print("AssertionError :", e)
```

Sortie

```
AssertionError : Complétez NOM
```

f-strings : afficher proprement

`f"...{x}..."` insère la valeur de `x`. `{x:8}` réserve 8 caractères (pour aligner des colonnes), `{p:.0%}` affiche un nombre en pourcentage sans décimale.

```
mot, p = "pas", 0.4795
print(f"[{mot:8}] {p:.0%}")
print(f"{'très':8}|")
```

Sortie

```
[pas     ] 48%
très    |
```

2. Les ensembles (set)

Un **ensemble** contient des éléments **sans doublon** et **sans ordre**. Idéal pour les listes de mots vides.

Créer un ensemble, tester l'appartenance

`set(liste)` supprime les doublons. `x in ens` est très rapide. `set()` crée un ensemble vide (attention : `{}` crée un dictionnaire vide).

```
mots = ["le", "plat", "le", "bon", "plat"]
ens = set(mots)
print(len(mots), len(ens))
print("bon" in ens, "pas" in ens)
vide = set()
print(type(vide), type({}))
```

Sortie

```
5 3
True False
<class 'set'> <class 'dict'>
```

Opérations : `&`, `-`, `|`, `<=`

`A & B` intersection (dans les deux) · `A - B` différence (dans A mais pas dans B) · `A | B` union (dans l'un ou l'autre) · `A <= B` : A est-il inclus dans B ?

```
A = {"le", "la", "pas", "et"}
B = {"le", "pas", "très"}
print(sorted(A & B))
print(sorted(A - B))
print(sorted(A | B))
print({"le"} <= A)
print(sorted((A - {"pas"}) | {"resto"}))   # retirer puis ajouter
```

Sortie

```
['le', 'pas']
['et', 'la']
['et', 'la', 'le', 'pas', 'très']
True
['et', 'la', 'le', 'resto']
```

Trier et prendre les premiers

`sorted(ens)` renvoie une **liste** triée ; `[:n]` garde les n premiers éléments.

```
ens = {"pas", "aucun", "jamais", "rien", "ni"}
print(sorted(ens)[:3])
```

Sortie

```
['aucun', 'jamais', 'ni']
```

3. Listes et compréhensions

La **liste en compréhension** construit une liste en une ligne : `[expression for x in collection if condition]`.

Filtrer et transformer

On lit : « pour chaque `m` de `mots`, si la condition est vraie, garder `m.lower()` ».

```
mots = ["Le", "plat", "!", "est", "BON", "."]
print([m.lower() for m in mots if m.isalpha()])
```

Sortie

```
['le', 'plat', 'est', 'bon']
```

Double boucle : aplatir une liste de listes

Les `for` s'écrivent **dans le même ordre** que des boucles imbriquées.

```
avis = [["le", "plat"], ["très", "bon"]]
print([m for liste in avis for m in liste])
```

Sortie

```
['le', 'plat', 'très', 'bon']
```

Une liste de listes (une liste par avis)

On met une compréhension **dans** une autre : on obtient une liste par document.

```
avis = [["Le", "plat", "!"], ["Très", "bon", "."]]
print([[m.lower() for m in liste if m.isalpha()] for liste in avis])
```

Sortie

```
[['le', 'plat'], ['très', 'bon']]
```

Compter et additionner avec `sum`

`sum(1 for x in ... if cond)` compte les éléments qui vérifient la condition. `sum(len(l) for l in listes)` additionne les longueurs.

```
mots = ["le", "plat", "est", "bon"]
vides = {"le", "est"}
print(sum(1 for m in mots if m in vides))
print(sum(len(l) for l in [["a", "b"], ["c"]]))
```

Sortie

```
2
3
```

Ensemble en compréhension

Même syntaxe avec des accolades : les doublons disparaissent. Pratique pour la taille d'un vocabulaire.

```
avis = [["le", "plat", "est", "bon"], ["le", "plat", "est", "froid"]]
voc = {m for liste in avis for m in liste}
print(len(voc), sorted(voc))
```

Sortie

```
5 ['bon', 'est', 'froid', 'le', 'plat']
```

4. Compter avec Counter

`Counter` (module `collections`) compte les éléments d'une liste.

Fréquences et mots les plus fréquents

`Counter(liste)` → un dictionnaire mot → nombre. `.most_common(n)` → les n plus fréquents, sous forme de couples `(mot, nombre)`. Un mot absent vaut 0.

```
from collections import Counter
freq = Counter(["bon", "le", "bon", "plat", "le", "bon"])
print(freq)
print(freq.most_common(2))
print(freq["bon"], freq["froid"], len(freq))
```

Sortie

```
Counter({'bon': 3, 'le': 2, 'plat': 1})
[('bon', 3), ('le', 2)]
3 0 3
```

`update` : la fréquence documentaire

`c.update(collection)` ajoute les éléments au compteur. Avec un **ensemble**, un mot n'est compté qu'**une fois par avis** : on obtient le nombre d'avis qui contiennent le mot.

```
from collections import Counter
avis = [["bon", "bon", "plat"], ["bon", "froid"]]
df = Counter()
for liste in avis:
    df.update(set(liste))
print(df.most_common())
```

Sortie

```
[('bon', 2), ('plat', 1), ('froid', 1)]
```

5. Les dictionnaires

Un dictionnaire associe une **clé** à une **valeur**.

Créer, lire, parcourir

`d[cle]` lit une valeur, `d.items()` donne les couples (clé, valeur) pour une boucle.

```
ligne = {"mot": "serveurs", "lemme": "serveur"}
print(ligne["lemme"])
for cle, valeur in ligne.items():
    print(cle, "->", valeur)
```

Sortie

```
serveur
mot -> serveurs
lemme -> serveur
```

`setdefault` : regrouper des valeurs

`d.setdefault(cle, set())` renvoie la valeur de la clé, en la créant (ensemble vide) si elle n'existe pas. On peut donc enchaîner `.add(...)`.

```
groupes = {}
for mot, initiale in [("sale", "s"), ("salé", "s"), ("bon", "b")]:
    groupes.setdefault(initiale, set()).add(mot)
print(groupes)
```

Sortie

```
{'s': {'sale', 'salé'}, 'b': {'bon'}}
```

Dictionnaire en compréhension (filtrer)

`{k: v for k, v in d.items() if condition}` garde seulement certaines entrées.

```
groupes = {"s": {"sale", "salé"}, "b": {"bon"}}
plusieurs = {k: v for k, v in groupes.items() if len(v) >= 2}
for k, v in sorted(plusieurs.items()):
    print(k, sorted(v))
```

Sortie

```
s ['sale', 'salé']
```

6. Écrire des fonctions

`def`, `return`, paramètre par défaut

Une fonction **renvoie** son résultat avec `return` (sans `return`, elle renvoie `None`). Un paramètre avec `=` a une valeur par défaut.

```
def saluer(nom, langue="fr"):
    if langue == "fr":
        return "Bonjour " + nom
    elif langue == "en":
        return "Hello " + nom
    raise ValueError("langue doit être 'fr' ou 'en'")

print(saluer("Sara"))
print(saluer("Sara", "en"))
try:
    saluer("Sara", "es")
except ValueError as e:
    print("ValueError :", e)
```

Sortie

```
Bonjour Sara
Hello Sara
ValueError : langue doit être 'fr' ou 'en'
```

`lambda` : petite fonction en une ligne

`lambda x: expression` équivaut à une fonction qui renvoie l'expression.

```
est_court = lambda m: len(m) <= 3
print(est_court("pas"), est_court("jamais"))
```

Sortie

```
True False
```

7. spaCy : analyser un texte

spaCy découpe le texte en **tokens** et calcule pour chacun ses attributs (lemme, catégorie…).

Analyser un texte ou tout un corpus

`nlp(texte)` renvoie un `Doc` (une suite de tokens). `nlp.pipe(liste_de_textes)` analyse plusieurs textes plus vite ; on le convertit en liste.

```
doc = nlp_fr("Le plat n'est pas bon.")
print([t.text for t in doc])

docs = list(nlp_fr.pipe(["Très bon.", "Trop salé."]))
print(len(docs))
```

Sortie

```
['Le', 'plat', "n'", 'est', 'pas', 'bon', '.']
2
```

Les attributs d'un token

Attribut	Signification
`t.text`	le texte du token
`t.lower_`	le texte en minuscules
`t.is_alpha`	True si le token ne contient que des lettres
`t.is_punct` / `t.is_space`	True si ponctuation / espace
`t.lemma_`	le lemme (forme du dictionnaire)
`t.pos_`	la catégorie grammaticale (NOUN, VERB, ADJ, DET, AUX…)

⚠️ Avec le **tiret bas** : `t.lemma_` est le texte, `t.lemma` (sans _) est un nombre.

```
for t in nlp_fr("Les serveurs étaient aimables."):
    print(t.text, t.lower_, t.is_alpha, t.is_punct, t.lemma_)
```

Sortie

```
Les les True False le
serveurs serveurs True False serveur
étaient étaient True False être
aimables aimables True False aimable
. . False True .
```

La liste de mots vides de spaCy

`nlp.Defaults.stop_words` est un ensemble (507 mots en français). Il contient les formes avec apostrophe (`n'`, `l'`), comme le tokeniseur de spaCy.

```
sw_spacy = set(nlp_fr.Defaults.stop_words)
print(len(sw_spacy), "pas" in sw_spacy, "n'" in sw_spacy, "jamais" in sw_spacy)
```

Sortie

```
507 True True False
```

Garder un mot spécial en un seul token

`add_special_case` déclare une chaîne que le tokeniseur ne doit pas couper.

```
nlp_fr.tokenizer.add_special_case("<URL>", [{"ORTH": "<URL>"}])
print([t.text for t in nlp_fr("Voir <URL> vite")])
```

Sortie

```
['Voir', '<URL>', 'vite']
```

8. NLTK : mots vides, racines, distance

La liste de mots vides de NLTK

`stopwords.words("french")` renvoie une **liste** (157 mots) : convertissez-la en ensemble. Elle contient `n` **sans** apostrophe (tokeniseur différent de spaCy).

```
sw_nltk = set(stopwords.words("french"))
print(len(sw_nltk), "n" in sw_nltk, "n'" in sw_nltk)
```

Sortie

```
157 True False
```

Raciner avec Snowball (français)

Le raciniseur **coupe les suffixes** avec des règles, sans dictionnaire : le résultat n'est pas toujours un vrai mot.

```
stemmer_fr = SnowballStemmer("french")
for m in ["mangeait", "rapides", "livraison", "serveuse"]:
    print(m, "->", stemmer_fr.stem(m))
```

Sortie

```
mangeait -> mang
rapides -> rapid
livraison -> livraison
serveuse -> serveux
```

Raciner en anglais : Porter et Snowball

`PorterStemmer()` est l'algorithme historique ; `SnowballStemmer("english")` (Porter 2) est sa version améliorée.

```
porter = PorterStemmer()
snow_en = SnowballStemmer("english")
for m in ["friendly", "generously", "was"]:
    print(m, porter.stem(m), snow_en.stem(m))
```

Sortie

```
friendly friendli friend
generously gener generous
was wa was
```

Distance de Levenshtein

`edit_distance(a, b)` = nombre minimal d'insertions, suppressions et substitutions pour passer de a à b.

```
from nltk.metrics.distance import edit_distance
print(edit_distance("servise", "service"))
print(edit_distance("restaurent", "restaurant"))
```

Sortie

```
1
1
```

Le plus petit tuple avec `min`

`min` compare des tuples **élément par élément** : d'abord le 1er, puis le 2e en cas d'égalité. Mettre `-fréquence` fait gagner le mot le plus fréquent.

```
candidats = [(1, -3, "service"), (1, -9, "serveur"), (4, -20, "le")]
d, _, meilleur = min(candidats)
print(d, meilleur)
```

Sortie

```
1 serveur
```

9. pandas : construire et filtrer un tableau

Un tableau à partir d'une liste de dictionnaires

Chaque dictionnaire devient une ligne ; les clés deviennent les colonnes. `list(df.columns)` donne les noms de colonnes, `len(df)` le nombre de lignes.

```
import pandas as pd
lignes = [{"mot": "plats", "lemme": "plat"},
          {"mot": "bon", "lemme": "bon"},
          {"mot": "étaient", "lemme": "être"}]
df = pd.DataFrame(lignes)
print(df)
print(list(df.columns), len(df))
```

Sortie

```
       mot lemme
0    plats  plat
1      bon   bon
2  étaient  être
['mot', 'lemme'] 3
```

Lire une case et filtrer des lignes

`df.loc[ligne, "colonne"]` lit une case. `df[condition]` garde les lignes où la condition est vraie. Pour combiner : `|` (ou), `&` (et), avec des **parenthèses** autour de chaque condition.

```
import pandas as pd
df = pd.DataFrame([{"mot": "plats", "lemme": "plat"},
                   {"mot": "bon", "lemme": "bon"},
                   {"mot": "étaient", "lemme": "être"}])
print(df.loc[0, "lemme"])
print(df[(df["lemme"] != df["mot"]) | (df["mot"] == "bon")])
print(df[df["mot"] == "bon"].iloc[0]["lemme"])
```

Sortie

```
plat
       mot lemme
0    plats  plat
1      bon   bon
2  étaient  être
bon
```

10. Chaînes, expressions régulières et Unicode

Méthodes de chaînes utiles

`.lower()` minuscules · `.split()` découpe sur les espaces · `.strip()` enlève les espaces au bord · `.replace(a, b)` remplace · `"".join(liste)` recolle · `.isalpha()` que des lettres ?

```
t = "  Service TRÈS lent  "
print(t.strip().lower().split())
print("froid".replace("o", "0"), "-".join(["a", "b", "c"]), "abc".isalpha())
```

Sortie

```
['service', 'très', 'lent']
fr0id a-b-c True
```

`re.sub` : remplacer selon un motif

`re.sub(motif, remplacement, texte)`. Motifs utiles : `\S+` (suite sans espace), `\w+` (lettres/chiffres), `\s+` (espaces), `(\w)\1{2,}` (une lettre répétée 3 fois ou plus ; `\1` = la lettre capturée), `[...]` (un caractère parmi une liste ou un intervalle). Préfixez les motifs par `r`.

```
import re
t = "Trooop bien !!! voir https://site.ma  @ali"
t = re.sub(r"https?://\S+", "<URL>", t)
t = re.sub(r"@\w+", "<USER>", t)
t = re.sub(r"(\w)\1{2,}", r"\1\1", t)
t = re.sub(r"\s+", " ", t)
print(t)
```

Sortie

```
Troop bien !!! voir <URL> <USER>
```

Écrire une lettre par son code Unicode

`"\u0623"` désigne le caractère de code U+0623 (أ). `ord(c)` donne le code d'un caractère, `hex()` l'écrit en hexadécimal. Un intervalle `[\u064B-\u0652]` couvre tous les signes de vocalisation.

```
import re
print("\u0623" == "أ", hex(ord("ة")))
print(re.sub("[\u064B-\u0652]", "", "شُكْرًا"))   # supprime les voyelles brèves
print("الـمطعـــــم".replace("\u0640", ""))          # supprime le tatweel
```

Sortie

```
True 0x629
شكرا
المطعم
```

`str.translate` : remplacer plusieurs caractères d'un coup

`str.maketrans("abc", "xyz")` crée une table a→x, b→y, c→z ; `texte.translate(table)` l'applique.

```
table = str.maketrans("٠١٢٣٤٥٦٧٨٩", "0123456789")
print("السعر ١٥٠ درهم".translate(table))
```

Sortie

```
السعر 150 درهم
```

`html.unescape`

Transforme les entités HTML en caractères.

```
import html
print(html.unescape("Thé &amp; menthe &lt;3"))
```

Sortie

```
Thé & menthe <3
```

`unicodedata` : enlever les accents

`normalize("NFD", texte)` sépare chaque lettre de son accent (é → e + ◌́). `category(c)` vaut `"Mn"` pour un accent seul : on le retire.

```
import unicodedata
nfd = unicodedata.normalize("NFD", "é")
print(len("é"), len(nfd), [unicodedata.category(c) for c in nfd])
print("".join(c for c in unicodedata.normalize("NFD", "Très élevé")
              if unicodedata.category(c) != "Mn"))
```

Sortie

```
1 2 ['Ll', 'Mn']
Tres eleve
```

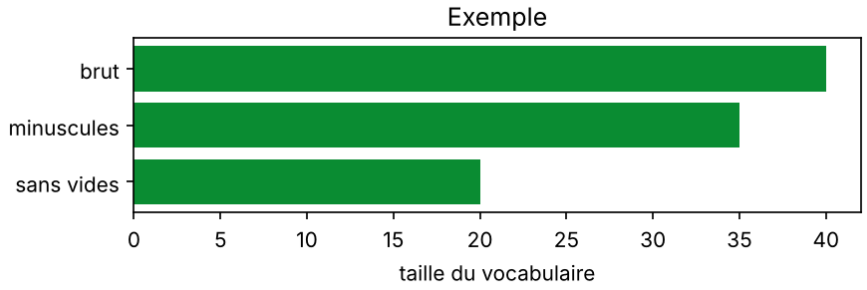
11. matplotlib : un diagramme en barres horizontales

`plt.barh`

`plt.barh(étiquettes, valeurs)` trace une barre par étiquette. `plt.gca().invert_yaxis()` met la première étiquette en haut. `xlabel` et `title` ajoutent les textes, `tight_layout` évite les coupures.

```
import matplotlib.pyplot as plt
plt.figure(figsize=(6, 2.2))
plt.barh(["brut", "minuscules", "sans vides"], [40, 35, 20], color="#0B8E36")
plt.gca().invert_yaxis()
plt.xlabel("taille du vocabulaire")
plt.title("Exemple")
plt.tight_layout()
plt.show()
```

Résultat



12. Pièges fréquents

La liste de mots vides doit correspondre au tokeniseur
NLTK contient `n`, spaCy produit `n'` : le token `n'` n'est pas retiré par la liste NLTK.

Retirer des mots vides peut effacer la négation
« Le plat est bon » et « Le plat n'est pas bon » donnent la même chose si `ne`, `n'`, `pas` sont retirés : retirez-les de la liste avec `-`.

`t.lemma_` et non `t.lemma`
Sans le tiret bas, on obtient un nombre (identifiant interne).

Une fonction sans `return` renvoie `None`
Les tests échouent alors avec un message du type « None != [...] ».

`{}` est un dictionnaire vide
Pour un ensemble vide, écrivez `set()`.

Parenthèses dans les filtres pandas
`df[(a) | (b)]` : sans parenthèses, Python lève une erreur.

Exécuter les cellules dans l'ordre
En cas d'erreur bizarre : *Exécution → Redémarrer et tout exécuter*.